3D: Solutions to the SVM student exercises

Problem statement and worked resolution

Companion to `03_D_student_exercises.ipynb`

How to use this explanation. This answer key gives the five missing snippets, explains their behavior, and develops checks students can use independently. The original worksheet remains unsolved; completed code was executed separately. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

The common model interface

A support vector classifier estimates a decision function from labeled examples. A pipeline combines the feature transformation and classifier into one fitted object. This matters because the learned boundary assumes a specific coordinate system: changing scaling at prediction time changes the question being asked.

`StandardScaler` learns one mean and scale per feature. `make_pipeline` creates sequential named steps; here the fitted classifier is accessible as `named_steps["svc"]`. The pipeline's `fit` first learns scaling on training rows, transforms those rows, and fits the SVC. Its prediction methods transform new rows with the stored means and scales before calling the classifier. Test targets are never part of these operations.

1 Exercise 1: separate two overlapping groups

1.1 Problem and data

The supplied generator places 400 points in two Gaussian clouds centered at $(-1.5, 0)$ and $(1.5, 0)$, with within-cloud standard deviation 1. The labels are 0 and 1. Split into 320 training and 80 test rows, stratifying by label. The clusters overlap, so even a reasonable linear boundary need not classify every row correctly.

A linear SVM learns $f(z) = w^T z + b$ in standardized coordinates and uses a sign decision. It balances a coefficient-norm penalty with hinge losses; C=1 is fixed for this exercise rather than tuned on test data. The library handles label encoding internally; manual hinge calculations would convert the labels to $-1, +1$.

1.2 TODO 1: create and fit the pipeline

Replace the first intentional error line with:

```
linear_model = make_pipeline(StandardScaler(), SVC(kernel="linear", C=1))
linear_model.fit(X_train, y_train)
```

The first line creates unfitted steps; the second learns their state. The input has shape $(320, 2)$, and the target has one label per row. Do not add an intercept column: SVC learns its own score offset. Calling `fit` again with test rows would train on the evaluation set, rather than measure generalization.

1.3 TODO 2: predict labels

```
linear_prediction = linear_model.predict(X_test)
```

This returns 80 labels without learning new statistics. Replacing this with `decision_function` would return scores instead of labels and would require an explicit threshold before using classification metrics.

The completed exercise gives 0.925 test accuracy, or 74 correct labels out of 80. The confusion matrix is

$$\begin{pmatrix} 40 & 0 \\ 6 & 34 \end{pmatrix},$$

with actual labels as rows and predictions as columns. The model has 80 training support vectors. This does not mean 80 training errors, and it has no direct relationship to the coincidentally equal test-set size. Support indices refer to rows contributing nonzero dual coefficients in the training fit.

1.4 Answers and a practical check

Test rows must use the training scaler because the classifier's weights and supporting points were learned in that standardized coordinate system. Refitting a test scaler makes the same raw input correspond to different coordinates. A support vector can be correctly classified on or inside the unit margin, so support does not mean error.

One way to inspect the model is to obtain `linear_model.named_steps["svc"].coef_` and its intercept. These describe standardized units. To report a boundary in original units, divide each slope by its feature's training scale and subtract the mean-adjustment from the intercept. Plotting standardized coefficients directly against raw data without this conversion can produce a misleading line.

2 Exercise 2: separate an inner and outer ring

2.1 Problem and representation

The supplied `make_circles` call creates 500 noisy points on rings with radius ratio 0.4 and noise scale 0.08. Training and test contain 400 and 100 rows. In this generator, the outer and inner rings carry distinct labels. A single straight line cannot enclose the inner ring while excluding the outer one on every side.

The radial basis function kernel is

$$K(z, v) = \exp(-\gamma \|z - v\|^2).$$

For gamma=1, points at standardized distance 1 have similarity $e^{-1} \approx 0.368$; at distance 2 their similarity is $e^{-4} \approx 0.0183$. The classifier can combine many local similarities into a curved score boundary. This does not mean it explicitly fits a circle equation: the shape comes from the learned support-vector combination.

2.2 TODO 3: create and fit the RBF pipeline

```
rbf_model = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1, gamma=1))
rbf_model.fit(X_ring_train, y_ring_train)
```

C and gamma are fixed teaching settings. Gamma changes the distance scale, while C changes the penalty for margin violations. Neither is the number of training iterations or the desired number of support vectors. Increasing gamma can model finer structure but can also fit noise; it does not monotonically improve held-out performance.

2.3 TODO 4: compute decision scores

```
ring_scores = rbf_model.decision_function(X_ring_test)
```

For binary labels 0 and 1, this is a vector of 100 real scores. Positive scores favor class 1; negative scores favor class 0. The values are not bounded by zero and one. A score of 0.4 cannot be read as a probability of 40%, and a score of -0.4 is not a negative probability.

2.4 TODO 5: turn scores into decisions

```
ring_prediction = (ring_scores >= 0).astype(int)
```

The comparison returns Boolean values, and integer conversion gives labels 0 and 1. This explicit rule assigns exactly zero to class 1. Library tie conventions need not be assumed identical in every implementation, so state the decision rule when comparing custom code at exact ties.

The completed run obtains perfect accuracy on the 100 held-out ring points:

$$\begin{pmatrix} 50 & 0 \\ 0 & 50 \end{pmatrix}.$$

There are 35 training support vectors. This sample is well separated at the chosen noise level, but a perfect finite test result does not prove zero population error. It also does not show that this kernel or these settings are universally preferable to a linear model on other data.

2.5 Answers to the written questions

The RBF combination can represent a curved boundary around the inner ring, unlike a single linear separator. Score -0.4 predicts label 0 under the specified rule. Increasing gamma makes similarities more local and may increase sensitivity to individual training observations; a new tuning study would require validation or CV, not repeated selection using this test set.

Documentation and debugging advice

The official SVM documentation describes the distinction between scores, labels, and separately estimated probabilities. Check the installed version when looking up constructor options, and inspect `classes_` rather than assuming a positive score always means your application's chosen event.

1. Replace the raised error line itself. Code below an intentional exception does not run.
2. Run setup and data generation before fitting; restart and run all cells after completing the worksheet.
3. A single input still needs two-dimensional shape $(1, 2)$, not a flat feature vector when calling a matrix-based estimator.
4. Preserve the pipeline when reusing a model. Saving only the SVC step drops its required transformation.
5. Distinguish `C`, `gamma`, support-vector count, and optimization tolerance; they play different roles.
6. Report the test denominator with accuracy, especially when a small sample produces a perfect score.

The supplied evaluation is deliberately simple. The worked notebooks add CV, kernel-equivalence checks, and cost-sensitive threshold selection once these basic fitting and prediction operations are understood.